Async APIs for Long Jobs

202 Accepted, job status, polling vs callbacks

Watch the 3-min video

In one line: for work that takes minutes, accept the request, enqueue the job and return `202 Accepted` with a job ID and a status URL; the job becomes a resource clients poll or get a webhook for.

1 Why you can't do it inside the request

A user uploads a **300-page PDF** to a document ingestion API. Parsing, chunking, embedding and indexing take about **ten minutes**. You can't hold an HTTP request open that long:

- **Load balancers and proxies time out** idle requests, often after 60 seconds by default.
- **Mobile connections drop** when the phone changes network or sleeps.
- **A single retry starts the whole job again**, doubling the cost and maybe the data.

So don't do the work in the request. **Validate** the input, **store** the file, **create a job record**, **put a message on a queue**, and immediately return `202 Accepted`. The body holds a job ID, and a `Location` header points to the status endpoint. The response takes milliseconds; the real work happens in the background.

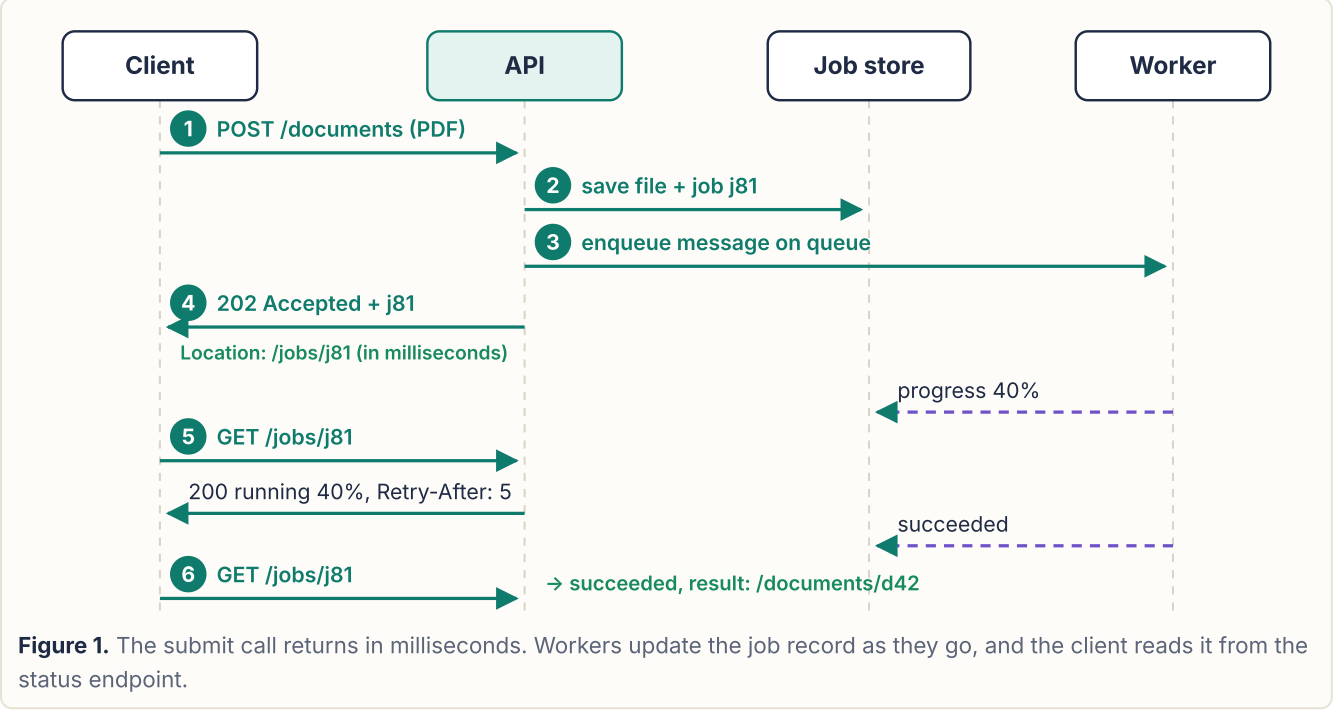


Figure 1. The submit call returns in milliseconds. Workers update the job record as they go, and the client reads it from the status endpoint.

2 The job is a resource: an operation

The job is a resource of its own, usually called an **operation**. It has an **ID**, a **status** (queued, running, succeeded, failed or cancelled), a **progress** percentage and **timestamps**. When it is done it holds either a **link to the result** or an **error**. Google's API guidelines call this a **long-running operation**; Azure uses the same pattern with an `Operation-Location` header.

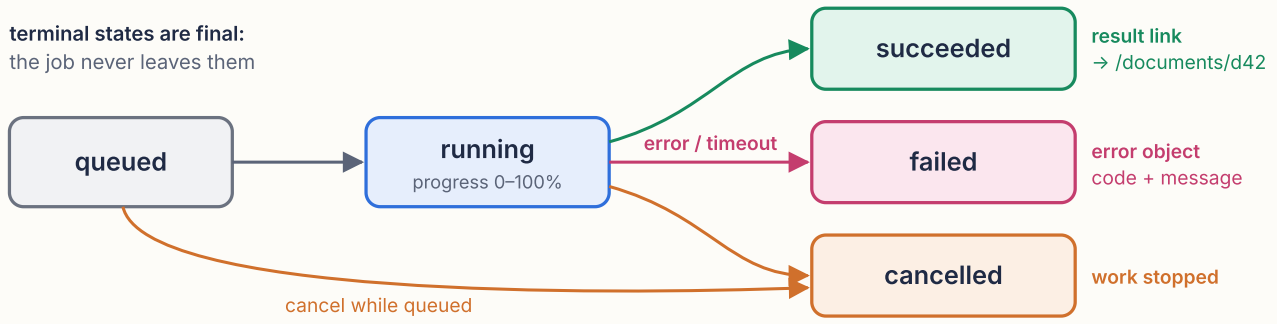


Figure 2. The job's lifecycle. Every job ends in exactly one terminal state, and a timeout turns a stuck job into a failed one.

```
HTTP/1.1 202 Accepted
Location: /jobs/j81
{"job_id": "j81", "status": "queued"}

GET /jobs/j81  →  200 OK
Retry-After: 5
{"id": "j81", "status": "running", "progress": 40,
 "created_at": "2026-11-04T10:00:00Z", "updated_at": "2026-11-04T10:04:10Z",
 "result": null, "error": null}
```

3 Polling or callbacks

Polling. The client sends `GET /jobs/j81` and gets the current status. A `Retry-After` header tells it how often to ask, say every **5 seconds**. Keep the status endpoint cheap: read from a fast store (a cache or a small indexed table), never recompute anything. When the job succeeds, the response links to the result.

Callbacks. The client passes a **callback URL** when it submits, and the server sends a **webhook** when the job finishes, **signed and retried** like any webhook. Browsers can subscribe with **server-sent events** or **WebSockets** for live progress. Many APIs offer both: callbacks for efficiency, polling as a reliable fallback when a webhook is lost.

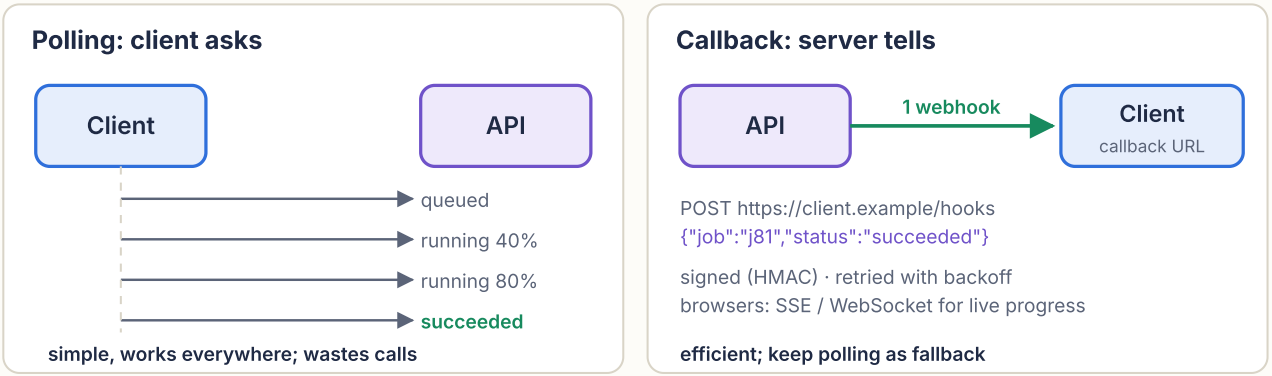


Figure 3. Polling needs nothing from the client but costs repeated calls. A callback is one message, but the client must run a reachable endpoint and verify signatures.

4 Make it robust

- **Idempotent submit.** The client sends an **idempotency key** with the upload. A retry with the same key returns the same job instead of creating a second one.
- **Checkpoint progress.** Workers save progress after each stage, so a crash resumes from the last checkpoint instead of starting over.

- **Cancellation.** Offer `POST /jobs/j81:cancel` or `DELETE /jobs/j81`; workers check the flag between stages.
- **Retention.** Expire old job records after a retention period (for example 7 or 30 days).
- **Timeouts.** Put a deadline on every job, so a stuck one is marked **failed** instead of "running" forever.

5 Example, end to end

WORKED EXAMPLE

`POST /documents` with the PDF returns **202** and job **j81**. The client polls the job: **queued**, then **running at 40%**, then **succeeded**, with a link to the new document. Meanwhile workers parse, chunk, embed and index, updating progress after each stage.

Cost of polling: **10,000** jobs in flight, each polled every 5 s, is 10,000 / 5 = **2,000 status requests per second**. A 10-minute job polled every 5 s makes about **120** calls; with a webhook it is **1** call. That is why the status read must be cheap, and why callbacks are worth offering.

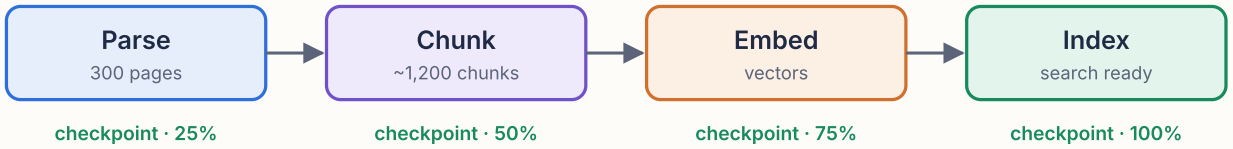


Figure 4. Each worker stage saves a checkpoint and updates progress. If the worker dies during embedding, a new one restarts from the end of chunking.

COMMON MISTAKES

- Doing ten minutes of work inside the request and raising every timeout to make it "work".
- Returning `200 OK` on submit: the work hasn't been done yet, so `202` is the honest answer.
- No idempotency key: a retried upload creates two jobs and two documents.
- A status endpoint that queries the heavy pipeline; polling traffic then slows the workers down.
- No timeout: jobs whose worker crashed show "running" forever.

6 Key terms

202 Accepted: request taken, work not done yet

Polling: client repeatedly GETs the status

Idempotency key: makes a retried submit return the same job

Location: header pointing to the status URL

Retry-After: header saying when to ask again

Checkpoint: saved progress a worker can resume from

Operation: resource that represents a long job

Callback / webhook: server calls the client when done

Retention: how long job records are kept

7 Interview questions

> **Q1. How do you design an API for a task that takes ten minutes?** BASIC
> Validate, store the input, create a job record, enqueue the work and return 202 Accepted with a job ID and a Location header for the status URL. Expose the job as an operation with status, progress, and a result or error. Clients poll (guided by Retry-After) or get a webhook.

Q2. Why 202 and not 200 or 201? BASIC

200 or 201 would claim the work is done or the resource exists. 202 says “accepted for processing, not finished”. The status endpoint later returns 200 with the job's state, and links to the created result.

Q3. What fields does a job status resource need? BASIC

ID, status (queued, running, succeeded, failed, cancelled), progress percentage, created and updated timestamps, and when finished either a result link or an error with code and message.

Q4. Mobile clients retry the upload when the network drops and you see duplicate jobs. Fix it. SCENARIO

Make submit idempotent: the client sends an idempotency key; the server stores key → job ID (e.g. for 24 h) and returns the existing job j81 on a retry instead of enqueuing a second one.

Q5. 50,000 jobs run at once and clients poll every second. What happens and what do you change? SCENARIO

That is 50,000 status reads per second. Send Retry-After: 5 to cut it to 10,000/s, serve status from a cache or small indexed table, and offer webhooks or SSE so most clients don't poll at all.

Q6. A worker crashes 8 minutes into a 10-minute job. What should happen? SCENARIO

The queue redelivers the message after the visibility timeout, and the new worker resumes from the last checkpoint (say after chunking) instead of re-parsing all 300 pages. A job timeout marks it failed if retries run out.

Q7. If you offer webhooks, why keep polling? TRICKY

Webhooks can be lost: the client endpoint is down, a firewall blocks it, or retries run out. Polling is the reliable fallback, and lets a client recover state after it restarts.

Q8. A job has shown “running” for 3 hours; normal jobs take 10 minutes. What design is missing? TRICKY

A per-job timeout or heartbeat. Workers should update a heartbeat; a sweeper marks jobs with no update for, say, 30 minutes as failed (or re-queues them), so clients get a final answer.

Next: SD-C-022 · API Contracts & OpenAPI backendroadmap.com